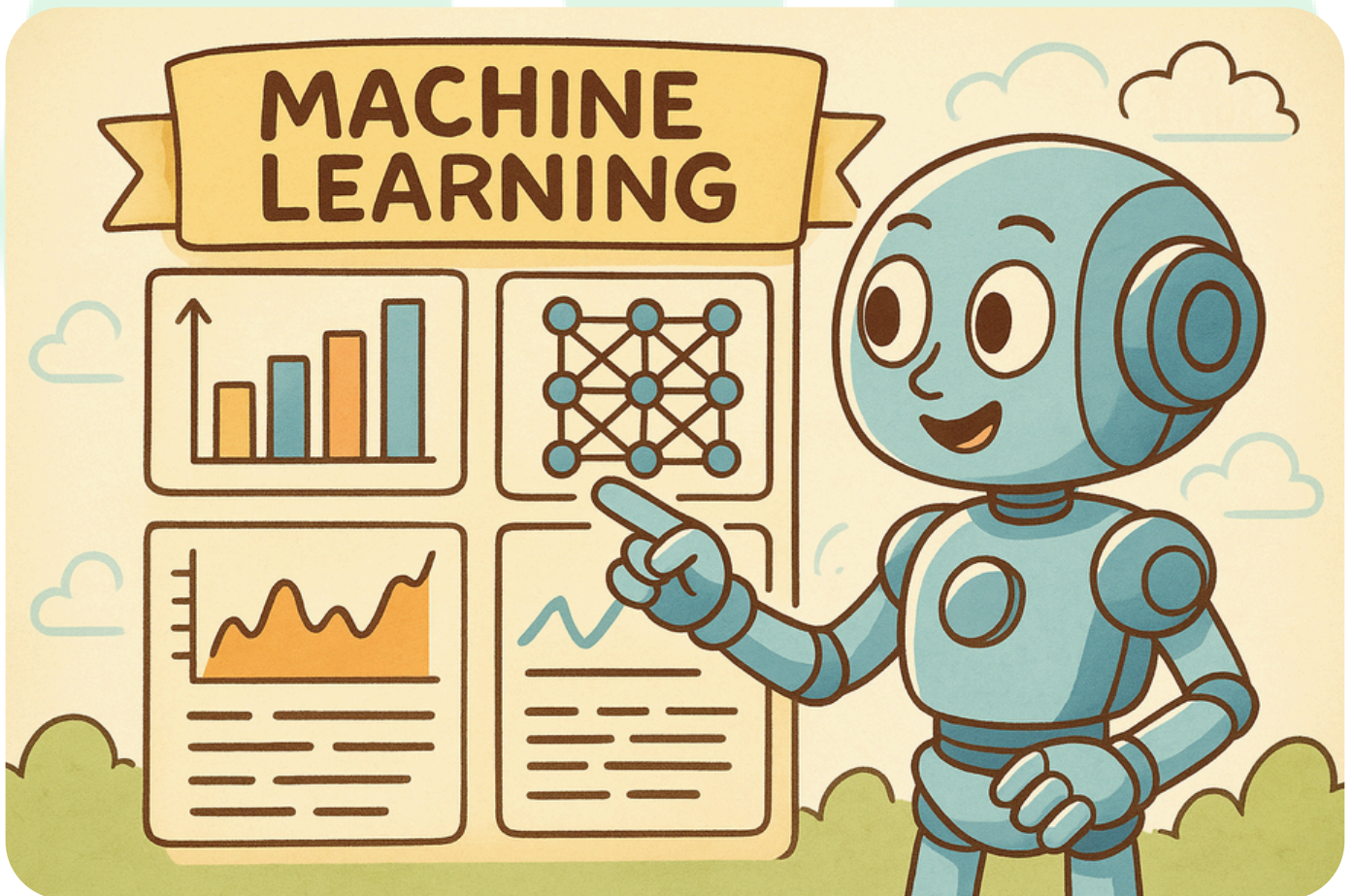


Machine learning

STUDY MATERIAL



Machine learning

MODULE

1

What is Machine Learning?

Machine Learning (ML) is a subfield of Artificial Intelligence (AI) that allows computers to learn patterns from data and make predictions or decisions without being explicitly programmed.

Imagine teaching a child to recognize apples and oranges:

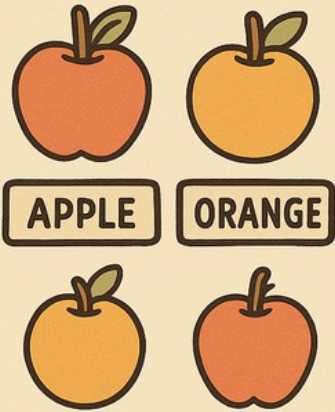
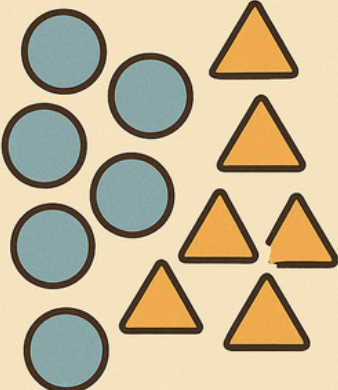
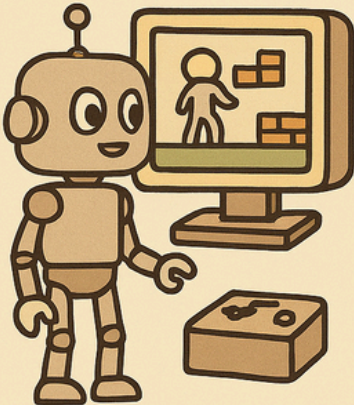
- You show examples of each fruit.
- The child observes features like color, shape, and size.
- Eventually, the child learns to identify new fruits without help.

ML does the same, but with data instead of physical objects.

Key components of Machine learning

1. Data – Information (numbers, text, images, etc.) used to train the model.
2. Features – Specific attributes in the data (e.g., height, weight, price).
3. Labels/Targets – The value or category we want to predict (e.g., spam or not spam).
4. Model – A mathematical function that maps input (features) to output (prediction).
5. Training – Teaching the model using historical data.
6. Inference – Using the trained model to make predictions on new data.
7. Evaluation – Checking how well the model performs.

Types of Machine Learning?

SUPERVISED LEARNING	UNSUPERVISED LEARNING	REINFORCEMENT LEARNING
		
CLASSIFICATION	CLUSTERING	GAMES

✓ 1. Supervised Learning

Definition: The model learns from labeled data (i.e., each input has a known output).

Examples:

- Email spam detection (Spam or Not)
- House price prediction (Price in \$)

Subtypes:

- Regression – Predict continuous values (e.g., price, temperature)
- Classification – Predict categories (e.g., male/female, cat/dog)

✓ 2. Unsupervised Learning

Definition: The model learns from unlabeled data, finding hidden patterns or structures.

Examples:

- Customer segmentation
- Market basket analysis

Subtypes:

- Clustering – Group similar items (e.g., customers with similar behavior)
- Dimensionality Reduction – Reduce the number of features while preserving info (e.g., PCA)

✓ 3. Semi-Supervised Learning

Definition: Uses a small amount of labeled data and a large amount of unlabeled data.

Use Case:

- Labeling medical data is expensive, so ML learns from some labeled scans and a lot of unlabeled ones.

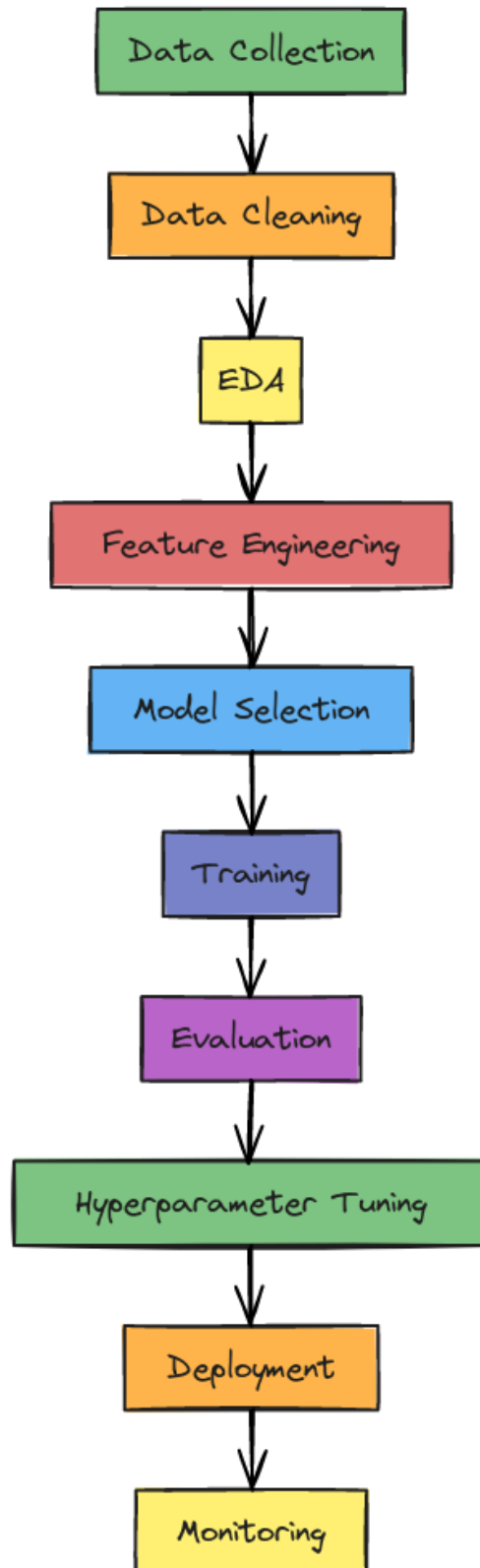
✓ 4. Reinforcement Learning (Advanced)

Definition: Model learns by trial and error in an environment to maximize rewards.

Example:

- Teaching a robot to walk
- Training AI to play games (like Chess or Go)

Flow of a typical Machine learning Project

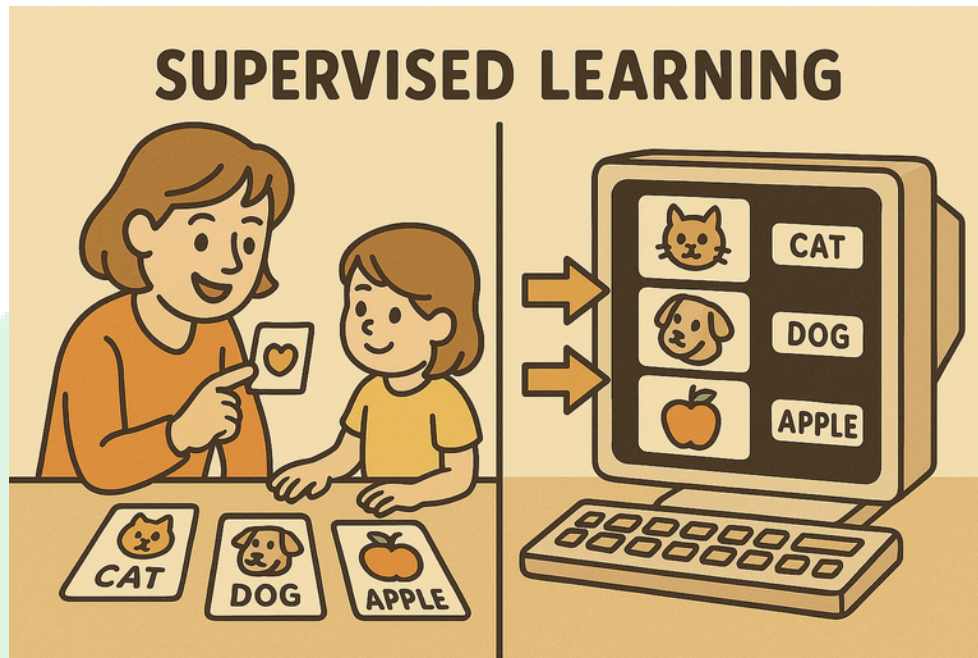


Supervised Learning

MODULE

2

What is Supervised Learning?



Supervised learning is a type of machine learning where:

- We are given input-output pairs.
- The model learns a function that maps inputs X to outputs Y .
- The goal is to minimize the error between predicted and actual values.

Imagine you're learning how to identify fruits. Someone shows you a fruit 🍏 and tells you, "This is an apple." Then another: 🍌 "This is a banana."

That person is **supervising** your learning — giving you **examples** (data) **with answers** (labels).

Once you've seen enough examples, you can predict on your own:

"Ah, I think this yellow curved fruit is a banana!"

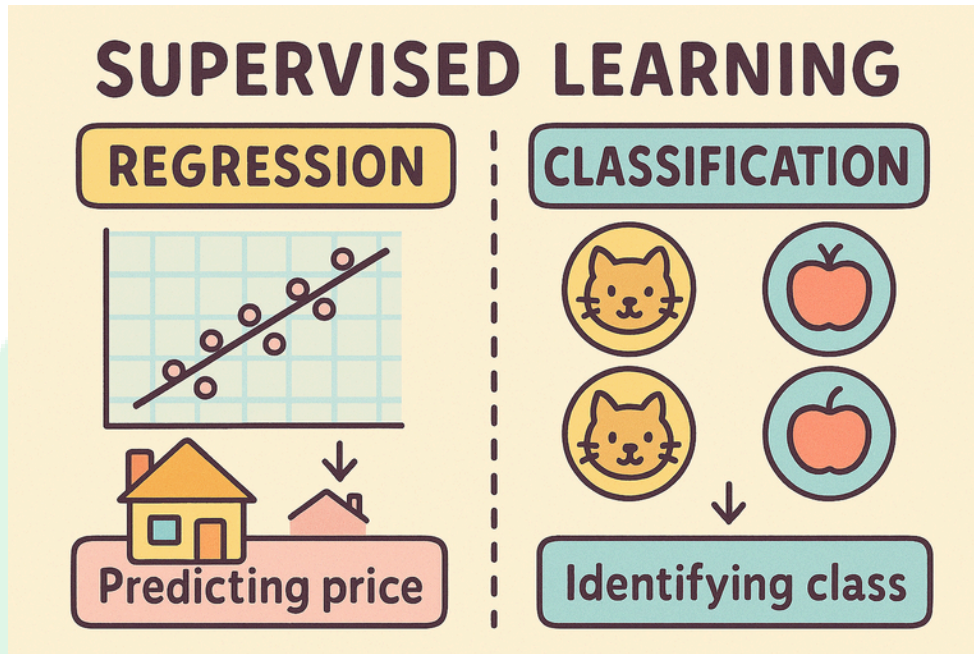
That's supervised learning:

Learning from labeled examples to make predictions.

It includes tasks like:

1. **Classification** (categorical output) — e.g., spam or not spam
2. **Regression** (numerical output) — e.g., predicting house price

Types of Supervised Learning?



Supervised learning can be broadly divided into two main types:

- **Regression** – This is used when the output variable is continuous. The model learns from labeled data to predict numeric values.

Example: Predicting house prices based on features like size, location, and number of rooms.

Common algorithms: Linear Regression, Polynomial Regression, Decision Trees for regression, Random Forest Regression.

- **Classification** – This is used when the output variable consists of discrete categories. The model learns from labeled data to assign inputs to predefined classes.

Example: Identifying whether an email is spam or not, classifying images into categories like cats vs. dogs.

Common algorithms: Logistic Regression, Decision Trees for classification, Random Forest, Support Vector Machines (SVM), Neural Networks.

Features (X)	Label (Y)
Email content	Spam / Not Spam
Hours studied	Exam score
Customer data	Will buy or not?

Input (Features/X) and Output (Labels/Y)

- Features (X): Vector or matrix of input variables.
 - Could be numeric, categorical, text, images, etc.
- Labels (Y): Ground truth output variable.
 - In regression: real numbers (e.g., price)
 - In classification: categories (e.g., "Yes", "No")

Shape:

- X is usually shape (n_samples, n_features)
- Y is shape (n_samples,)

In machine learning:

- X = input data, independent variables, features
- Y = correct answer, dependent variable, label

X1 (Age)	X2 (Income)	Y (Will Buy?)
25	50,000	Yes
40	30,000	No

Code:

```

import pandas as pd

# Input features
X = pd.DataFrame({
    'Age': [25, 40, 30],
    'Income': [50000, 30000, 45000]
})

# Labels
Y = pd.Series(['Yes', 'No', 'Yes'])

print(X)
print(Y)
  
```

Training and Testing Sets

- The dataset is split into:
 - **Training Set:** Used to **fit** the model.
 - **Testing Set:** Used to **evaluate** model performance.
- Common split: **80% train / 20% test**

Sometimes you also use:

- **Validation Set:** for tuning parameters

When you study for an exam, you:

- Learn using textbooks and practice problems → Training 📖
- Test yourself with a mock exam → Testing 📝

Same with ML models:

- Training set: used to train the model
- Testing set: used to check how well the model performs on unseen data

Code:

```

from sklearn.model_selection import train_test_split

X = [[25, 50000], [40, 30000], [30, 45000], [22, 60000]]
Y = ['Yes', 'No', 'Yes', 'Yes']

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.25)

print("Train:", X_train, y_train)
print("Test:", X_test, y_test)
  
```

Model Representation: Function

$$f(X) \rightarrow Y$$

- Supervised learning learns a function f :

$$f : X \rightarrow Y$$

- The goal is to find f that minimizes the loss/error between predicted and true Y
- This function can be:
 - Linear (e.g., Linear Regression)
 - Nonlinear (e.g., Decision Tree, Neural Network)

The model is like a magic box 🧙♂️:

- You put in the input X
- It gives you a prediction Y
- The model learns a rule or pattern from data to do this

Example:

$$f(X) = X1 * 1000 + X2 * 2$$

Where X1 is age and X2 is income → outputs predicted score

Code:

```

from sklearn.linear_model import LinearRegression
import numpy as np

X = np.array([[25], [40], [30]])
Y = np.array([200, 150, 180]) # some score

model = LinearRegression()
model.fit(X, Y)

print("Prediction for age 35:", model.predict([[35]]))
  
```

Real-life Use Cases of Supervised Learning

1. Credit Scoring

- Predict whether a person will repay a loan
- Input (X): age, income, credit history
- Output (Y): Good Credit (1) or Bad Credit (0)

2. Email Classification

- Input: email text (converted to numbers using NLP)
- Output: Spam or Not Spam

3. Stock Price Prediction

- Predict the future price of a stock
- Input: past prices, volume, indicators
- Output: next day's price (numerical → regression)

Bonus: What Math is Involved?

For models like Linear Regression:

$$Y = w_1X_1 + w_2X_2 + \dots + w_nX_n + b$$

- w : weights (what the model learns)
- b : bias
- Model tries to minimize error:

$$\text{Loss} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

(Just like minimizing how far off your guesses are!)

Code:

- 1. Credit Scoring
- Predict whether a person will repay a loan
- Code:
- Input (X): age, income, credit history
-
- Output (Y): Good Credit (1) or Bad Credit (0)

Summary

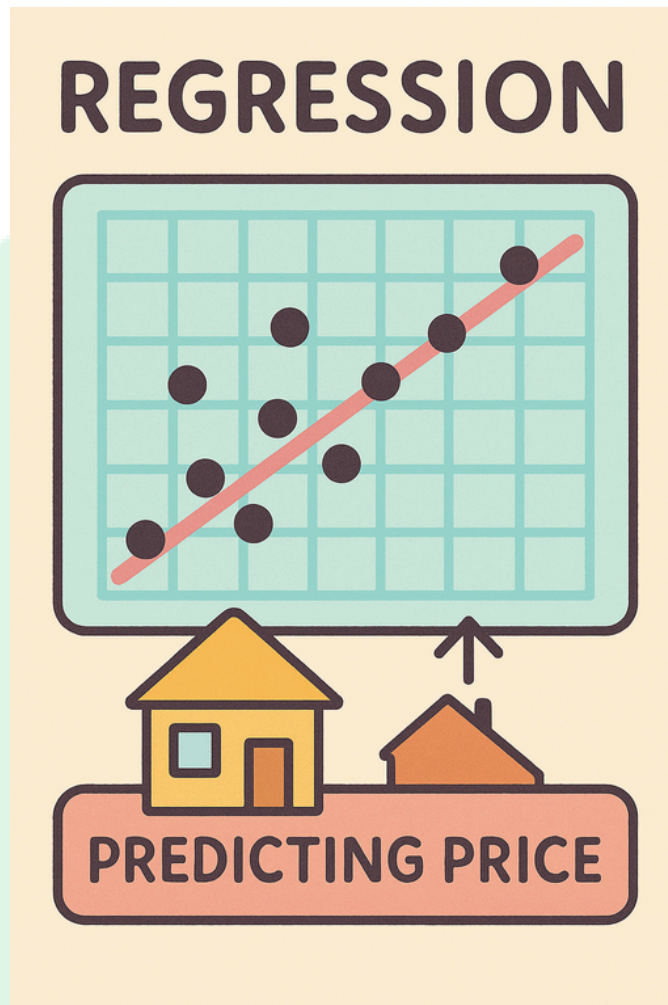
Concept	Simple View	Technical View
Supervised Learning	Learning from examples with answers	Mapping $X \rightarrow Y$ with labeled data
Features (X), Labels (Y)	Input data and correct output	Independent and dependent variables
Train/Test Sets	Practice vs. test exam	Split data to fit and evaluate the model
Model $f(X) \rightarrow Y$	A rule that converts input to prediction	A function learned from data
Real-life Examples	Credit, Email, Stock	Classification and regression problems

Regression

MODULE

3

What is Regression?



Regression is a supervised learning technique used to predict continuous numeric values (unlike classification, which predicts categories).

- The model estimates a function $f(X) \rightarrow Y$ to minimize prediction errors.

Regression is like guessing the marks a student might get based on the hours they studied.

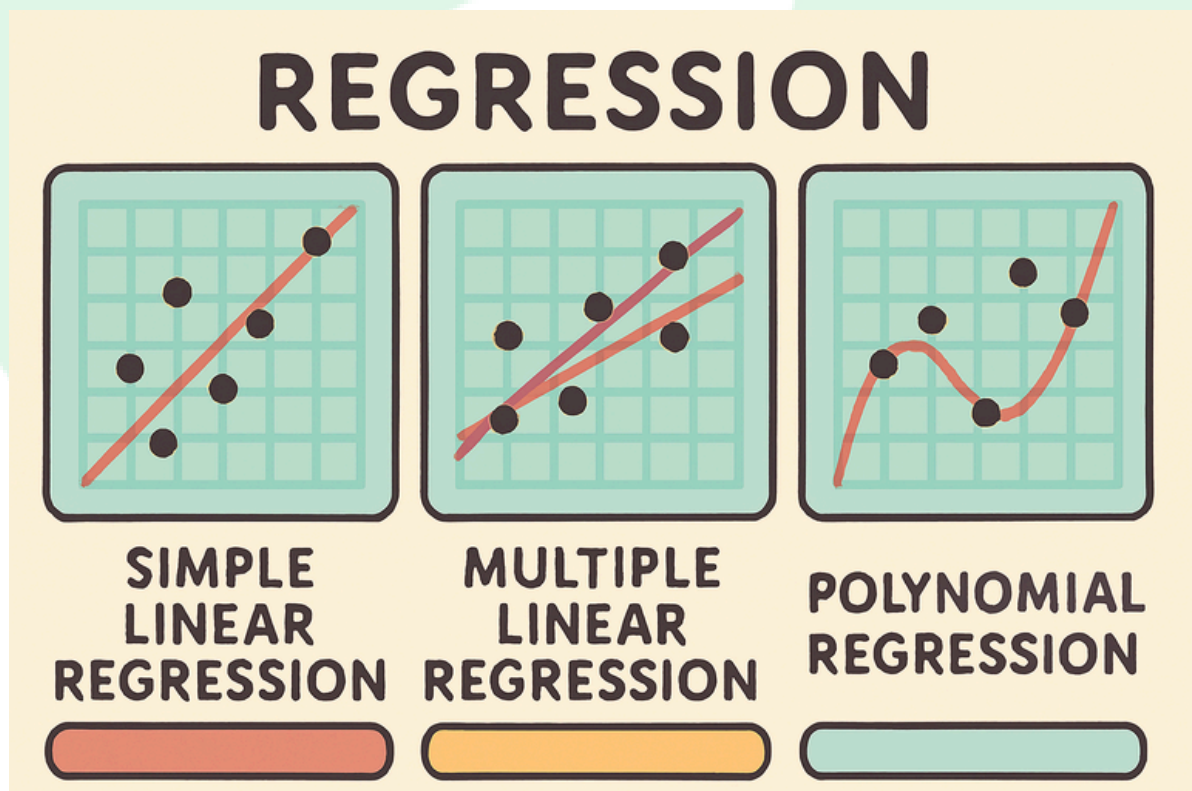
More hours studied $\rightarrow$ likely more marks 

The model draws a line or curve that best fits the data points.

Types of Regression?

Regression can be divided into different types based on the complexity of relationships they model:

1. **Simple Linear Regression** – Models the relationship between one independent variable and one dependent variable using a straight line.
 - Example: Predicting a student's test score based on study hours.
2. **Multiple Linear Regression** – Uses multiple independent variables to predict a continuous outcome.
 - Example: Estimating house prices based on factors like size, location, and number of rooms.
3. **Polynomial Regression** – Models complex relationships using curves rather than straight lines by introducing polynomial terms.
 - Example: Predicting population growth trends with fluctuating patterns.



Simple linear Regression

◆ Equation of a Line:

$$y = mx + c$$

- m: slope → how much y changes for one unit change in x
- c: intercept → value of y when x = 0

◆ Example:

Predict marks (y) based on hours studied (x):

```

from sklearn.linear_model import LinearRegression
import numpy as np

X = np.array([[1], [2], [3], [4]]) # hours
Y = np.array([50, 60, 65, 70])    # marks

model = LinearRegression()
model.fit(X, Y)

print("Predicted marks for 5 hrs:", model.predict([[5]]))
  
```

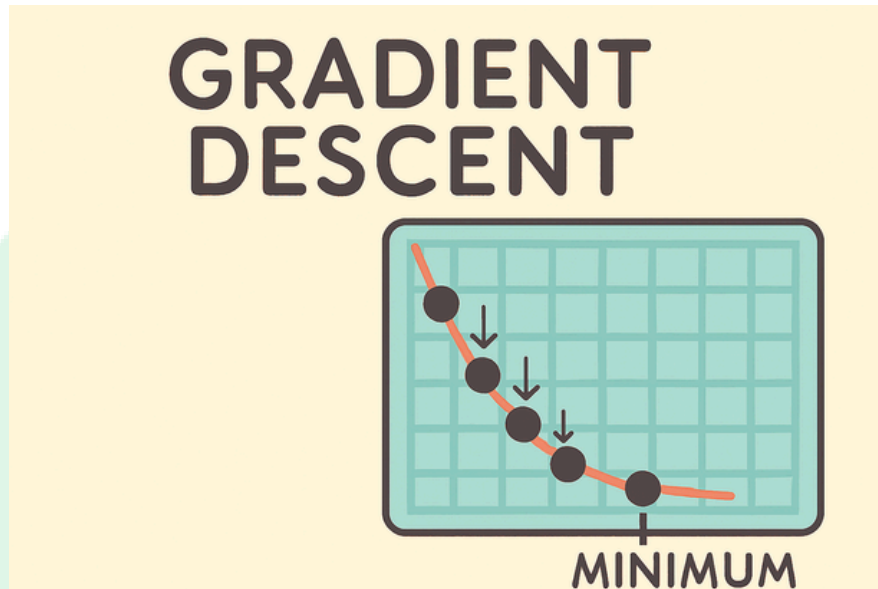
◆ Loss Function: Mean Squared Error (MSE)

$$Loss = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- Measures average squared difference between actual and predicted
- Lower MSE → better fit

◆ Gradient Descent:

- It's an algorithm to find the best m and c by minimizing the MSE
- Like going downhill on an error mountain to reach the lowest point



```

# Simplified gradient descent for learning
m, c = 0, 0
lr = 0.01

for i in range(1000):
    y_pred = m * X.flatten() + c
    dm = -2 * np.mean((Y - y_pred) * X.flatten())
    dc = -2 * np.mean(Y - y_pred)
    m -= lr * dm
    c -= lr * dc
  
```

◆ Assumptions of Linear Regression:

1. Linearity: Relationship between X and Y is linear
2. Homoscedasticity: Constant variance of errors
3. Independence: Observations are independent
4. Normality: Residuals are normally distributed
5. No multicollinearity (for multiple regression)

Multiple linear Regression

◆ Equation:

$$y = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$$

Multiple inputs → 1 output

Example: Predict house price using area, rooms, age

```

X = [[1000, 2], [1500, 3], [1200, 2]]
Y = [200000, 250000, 220000]

model = LinearRegression()
model.fit(X, Y)
  
```

◆ Multicollinearity:

When independent variables are highly correlated, it:

- Makes coefficients unstable
- Hard to interpret which feature is actually important

Solution:

- Remove one of the correlated features
- Use regularization

Polynomial Regression

If data doesn't form a line but a curve, linear regression won't work.
 Use powers of x to curve the model.

$$y = b_0 + b_1x + b_2x^2 + b_3x^3 + \dots$$

```

from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform([[1], [2], [3], [4]])

model = LinearRegression()
model.fit(X_poly, [2, 6, 14, 28])
  
```

◆ Multicollinearity:

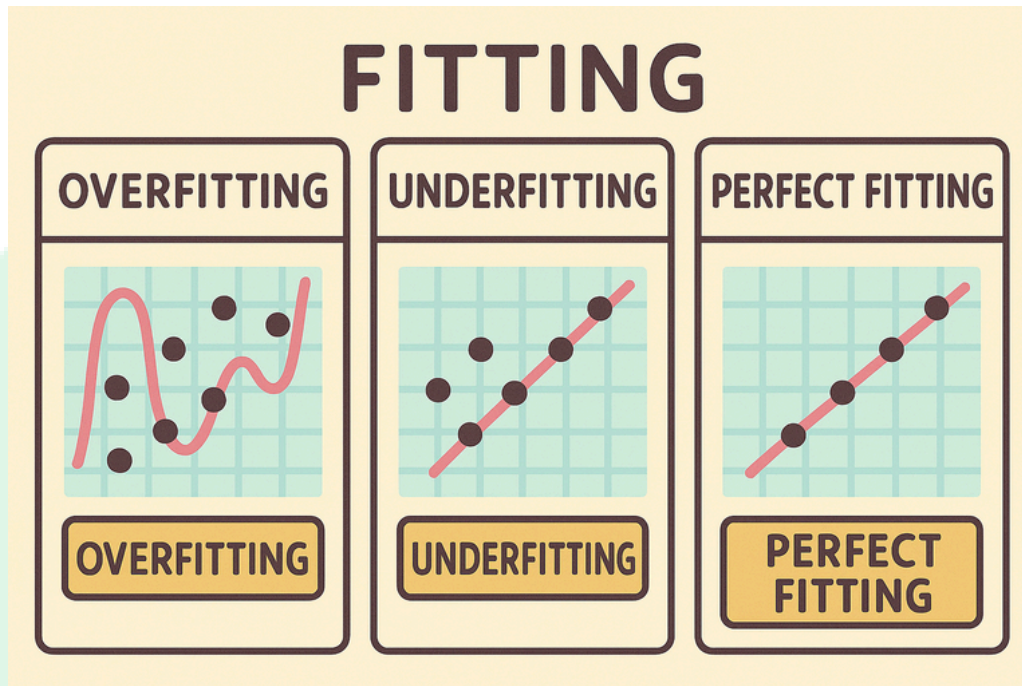
When independent variables are highly correlated, it:

- Makes coefficients unstable
- Hard to interpret which feature is actually important

Solution:

- Remove one of the correlated features
- Use regularization

What Are Overfitting and Underfitting?



◆ Simple Explanation (Real-life Analogy):

Imagine you're a student preparing for an exam:

- **Underfitting:** You barely study, only memorize a few basics → You can't answer even easy questions.
- **Overfitting:** You memorize every word of your textbook including typos → You struggle with the actual exam because the questions look slightly different.

What you really need is a balanced understanding — generalized learning.

Underfitting

✓ Simple Definition:

Model is too simple to learn the patterns in the data.

It doesn't learn enough → bad performance on training and testing data.

⚙️ Technical Definition:

- High bias (rigid assumptions)
- Can't capture underlying trend of data
- Happens when the model is:
 - Too simple (e.g., linear for curved data)
 - Not trained enough
 - Wrong features used

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# Data: curve
X = np.linspace(0, 10, 100).reshape(-1, 1)
y = np.sin(X).ravel()

# Linear model (too simple)
model = LinearRegression()
model.fit(X, y)
y_pred = model.predict(X)

plt.scatter(X, y, label='Actual data')
plt.plot(X, y_pred, color='red', label='Underfit model')
plt.legend()
plt.title("Underfitting Example")
plt.show()
  
```

Overfitting

✓ Simple Definition:

Model learns the data too well, including noise and random fluctuations. It performs very well on training data but poorly on new/unseen data.

⚙️ Technical Definition:

- High variance
- Model is too complex for the task
- Follows training data perfectly, fails to generalize

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

poly = PolynomialFeatures(degree=15) # very high degree
X_poly = poly.fit_transform(X)

model = LinearRegression()
model.fit(X_poly, y)
y_pred = model.predict(X_poly)

plt.scatter(X, y, label='Actual data')
plt.plot(X, y_pred, color='green', label='Overfit model')
plt.legend()
plt.title("Overfitting Example")
plt.show()
```


Summary

Aspect	Underfitting	Overfitting
Error Type	High bias	High variance
Model Simplicity	Too simple	Too complex
Performance	Poor on both training and test sets	Good on training, bad on test
Cause	Not enough learning	Learning noise and outliers
Fix	Add features, complexify model	Regularize, reduce complexity, more data

Tip: How to Detect It?

Use training vs testing error:

- If both are high → underfitting
- If training error is low but test error is high → overfitting
- If both are low → great model!

You can visualize this with a learning curve.

Solutions:

Problem	Fixes
Underfitting	Add features, increase model complexity, train longer
Overfitting	Regularization (Rid

Regularization Techniques

Used to reduce overfitting by penalizing large coefficients.

Regularization techniques help prevent overfitting by adding constraints to the model. Two popular methods are:

1. **Lasso Regression (L1 Regularization)** – Adds a penalty based on the absolute values of the coefficients, encouraging some of them to become zero. This helps with feature selection by eliminating less important variables.
 - Example: If predicting house prices, Lasso can remove irrelevant features like the color of the front door.
2. **Ridge Regression (L2 Regularization)** – Adds a penalty based on the squared values of the coefficients, preventing large values and ensuring smooth, stable predictions without eliminating features.
 - Example: If predicting student performance, Ridge ensures that all contributing factors (like study hours and sleep patterns) remain considered.

◆ Ridge Regression (L2 Regularization)

$$Loss = MSE + \lambda \sum w^2$$

- Penalizes squared weights
- Keeps all features but shrinks them

```

from sklearn.linear_model import Ridge
model = Ridge(alpha=1.0)
model.fit(X, Y)
  
```

◆ Lasso Regression (L1 Regularization)

$$Loss = MSE + \lambda \sum |w|$$

- Penalizes absolute value of weights
- Can eliminate irrelevant features by setting weights to 0

```
from sklearn.linear_model import Lasso
model = Lasso(alpha=0.1)
model.fit(X, Y)
```

◆ ElasticNet (Hybrid of L1 and L2)

$$Loss = MSE + \lambda_1 \sum |w| + \lambda_2 \sum w^2$$

- Combines strengths of both Ridge and Lasso

```
from sklearn.linear_model import ElasticNet
model = ElasticNet(alpha=0.1, l1_ratio=0.5)
```

Evaluation Metrics

◆ MAE (Mean Absolute Error)

$$MAE = \frac{1}{n} \sum |y_i - \hat{y}_i|$$

- Average absolute difference
- Easy to interpret, but less sensitive to large errors

◆ MSE (Mean Squared Error)

$$MSE = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$$

- Punishes large errors more
- Preferred when large errors are worse

◆ RMSE (Root Mean Squared Error)

$$RMSE = \sqrt{MSE}$$

- Same units as target variable
- Easier to interpret than MSE

◆ R² Score (Coefficient of Determination)

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}$$

- $R^2 = 1 \rightarrow$ perfect fit
- $R^2 = 0 \rightarrow$ model is no better than average
- Can be negative if worse than naive prediction

```

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

mae = mean_absolute_error(y_true, y_pred)
mse = mean_squared_error(y_true, y_pred)
rmse = mse ** 0.5
r2 = r2_score(y_true, y_pred)
  
```

Summary table

Topic	Concept	Key Notes
Simple Linear Regression	One input, one output	$y = mx + c$
Loss Function	MSE	Average squared error
Gradient Descent	Optimization method	Minimizes loss
Assumptions	Linearity, independence, normality	Important for model validity
Multiple Regression	Many inputs	$y = b_0 + b_1x_1 + \dots$
Multicollinearity	Inputs are correlated	Use regularization
Polynomial Regression	Curve fitting	Adds powers of x
Regularization	Avoid overfitting	L1 = Lasso, L2 = Ridge
Evaluation	MAE, MSE, RMSE, R^2	Measure model performance

What Are Bias and Variance?

◆ Bias:

Bias measures the error due to wrong assumptions in the learning algorithm.
 High Bias → Model is too simple, can't learn data patterns
 Causes underfitting

✗ Example: Using a straight line to fit curved data.

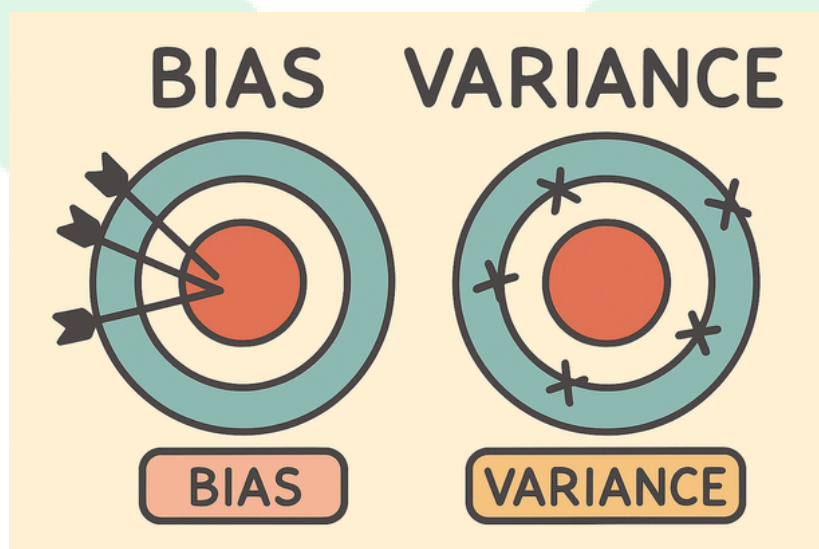
◆ Variance:

Variance measures the model's sensitivity to small fluctuations in the training data.
 High Variance → Model is too complex, learns noise
 Causes overfitting

✗ Example: A model that perfectly fits training data but performs poorly on test data.

🧠 Simple Analogy: Archery 🎯

- Bias: How far your arrows are from the bullseye on average.
- Variance: How scattered your arrows are around each other.



Arrow Pattern	Meaning
All arrows far from bullseye in one place →	High bias, low variance
Arrows scattered all over →	Low bias, high variance
Arrows tightly clustered around bullseye →	Low bias, low variance (Ideal)
Arrows far and scattered →	High bias, high variance (Worst)

Mathematically (Simplified):

Total prediction error =

$$Bias^2 + Variance + Irreducible Error$$

Where:

- Bias²: Error from incorrect model assumptions
- Variance: Error from sensitivity to data variation
- Irreducible Error: Noise we can't remove (random)

How to Handle Bias and Variance?

Problem	Fixes
High Bias	Use more complex models (e.g., polynomial, trees)
High Variance	Use simpler models, regularization (L1/L2), more data
Ideal Case	Balance both: low bias + low variance

Summary table

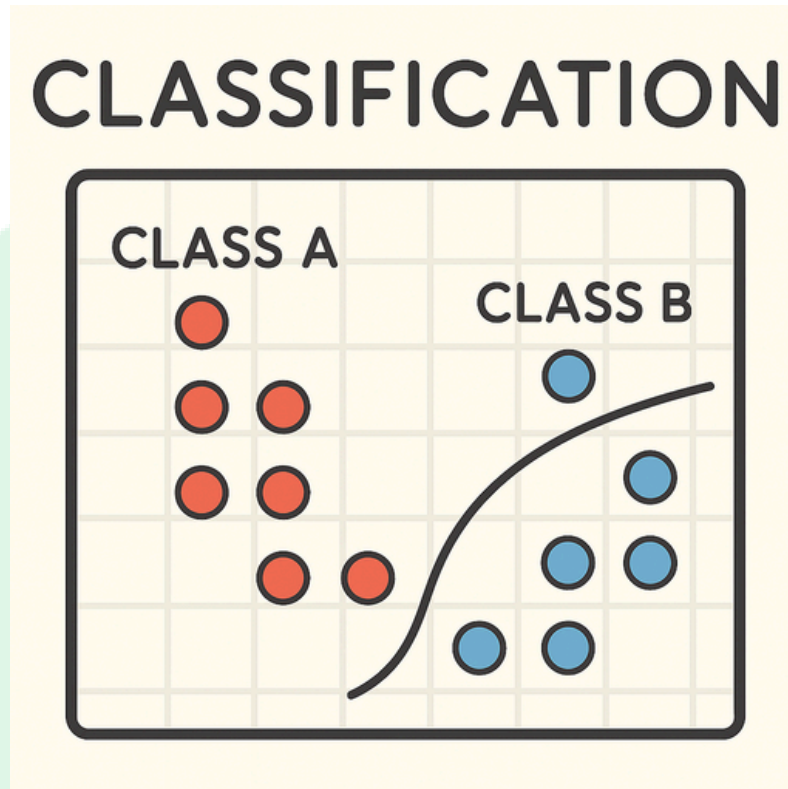
Concept	High Bias	High Variance
Error Source	Incorrect assumptions	Sensitivity to training data
Behavior	Misses patterns (underfits)	Captures noise (overfits)
Training Error	High	Low
Test Error	High	High
Fix	More complex model	Simpler model, regularization, more data

Classification

MODULE

4

What is Classification?



Classification is a **supervised learning** task where the output variable is **categorical** — meaning it takes on one of a **finite set of classes or labels**.

✓ Simple Explanation:

Classification is a type of machine learning where the goal is to **sort data into categories or labels**.

Think of it like a smart sorting machine:

- 📧 Email → Spam or Not Spam
- 🧬 Tumor → Cancerous or Benign
- 🛒 Product Review → Positive, Negative, or Neutral

You give the model some input (like text, images, or numbers), and it **predicts the correct category**.

Types of classification:

Type	Example
Binary	Spam vs. Not Spam (2 classes)
Multiclass	Dog, Cat, Horse (more than 2)
Multilabel	Email tagged as Work & Important

How it Works

1. Input (X): Features (e.g., age, salary, image pixels, text)
2. Model learns patterns from training data.
3. Prediction (Y): Output is a category/label, not a number.

Real-life Examples

Application	Input	Output (Class)
Email Filtering	Email content	Spam or Not Spam
Medical Diagnosis	Patient data	Has Disease / No Disease
Credit Card Fraud Detection	Transaction details	Fraud / Not Fraud
Image Classification	Image pixels	Dog / Cat / Bird
Sentiment Analysis	Text review	Positive / Neutral / Negative

Classification vs Regression

Aspect	Classification	Regression
Output Type	Category (Discrete)	Number (Continuous)
Example	Spam or Not	Price of a house
Models Used	Logistic Reg., Decision Tree	Linear Reg., SVR
Evaluation	Accuracy, Precision, Recall	MSE, RMSE, R ² Score

Behind the Scenes

Most classification algorithms (like logistic regression, decision trees, or SVM) work by:

1. Calculating a score or probability
2. Choosing the class with the highest score

For **binary classification**, logistic regression uses the **sigmoid function** to output probabilities between 0 and 1.

Logistic Regression

Logistic Regression helps you predict categories — like Yes/No, Spam/Not Spam, or Pass/Fail.

Unlike linear regression (which predicts continuous numbers), logistic regression predicts probabilities that are converted into binary outputs (0 or 1).

🎯 Example: Will a person buy a product based on their age and income? Logistic regression will give you the probability of buying → You can classify that as "Yes" or "No" using a threshold (e.g., 0.5).

◆ Logistic Function (Sigmoid)

The Sigmoid function is the core of logistic regression. It transforms any number into a value between 0 and 1.

$$\text{Sigmoid}(z) = \frac{1}{1 + e^{-z}}$$

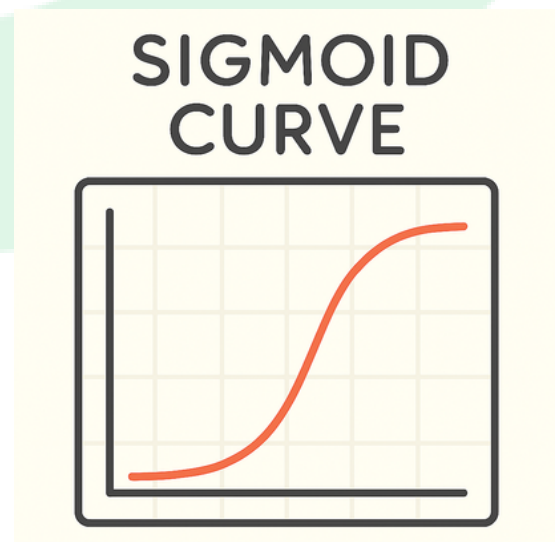
Where:

- $z = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$
- It's a linear combination of inputs, just like in linear regression.

This value represents the probability of the outcome being class 1.

Sigmoid Curve (Described Visually):

- S-shaped curve
- Steep around 0.5
- Flattens toward 0 and 1
- Converts raw output z into a value between [0, 1]



Thresholding

Once you get a probability (say 0.78), you convert it into a class:

Probability	Predicted Class
≥ 0.5	1 (e.g., Yes)
< 0.5	0 (e.g., No)

This threshold (default is 0.5) can be tuned based on the problem (e.g., fraud detection, cancer prediction).

Binary Cross-Entropy (Log Loss)

This is the loss function used to train logistic regression.

Formula:

$$\text{LogLoss} = -\frac{1}{N} \sum_{i=1}^N [y_i \cdot \log(\hat{y}_i) + (1 - y_i) \cdot \log(1 - \hat{y}_i)]$$

Where:

- y_i is the actual label (0 or 1)
- $\hat{y}_i$ is the predicted probability
- Lower log loss = better performance

✓ Simple Explanation:

- If the model predicts 0.9 and the truth is 1 → small error
- If the model predicts 0.1 and the truth is 1 → large error
- **It punishes wrong confident predictions more**

Code:

```

from sklearn.linear_model import LogisticRegression
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, log_loss

# Generate sample binary classification data
X, y = make_classification(n_samples=1000, n_features=2, n_redundant=0, random_state=42)

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)

# Train logistic regression model
model = LogisticRegression()
model.fit(X_train, y_train)

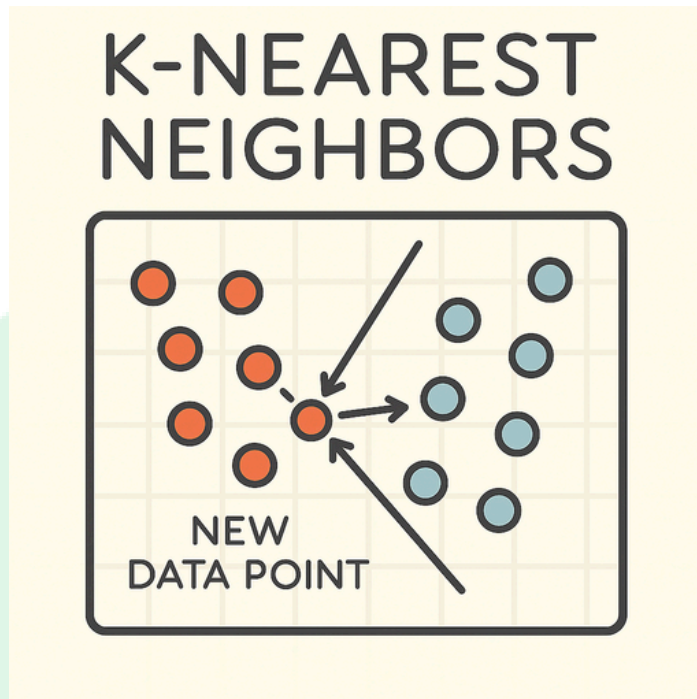
# Predict probabilities and classes
probs = model.predict_proba(X_test)[: , 1] # probabilities of class 1
preds = (probs >= 0.5).astype(int)         # apply threshold

# Evaluate
print("Accuracy:", accuracy_score(y_test, preds))
print("Log Loss:", log_loss(y_test, probs))
  
```

Real-life Use Cases of Logistic Regression

Use Case	Classes to Predict
Email Classification	Spam / Not Spam
Medical Diagnosis	Disease / No Disease
Credit Risk Scoring	Default / Not Default
Online Transaction	Fraudulent / Legitimate
Customer Churn	Will Churn / Won't Churn

K-Nearest Neighbors (KNN)



- KNN is a **supervised learning algorithm**
- It can be used for:
 - Classification (predict category)
 - Regression (predict number)
- It stores the entire training data and makes predictions only **during query time** (that's why it's called a **lazy learner**)

💡 Simple Explanation:

KNN is like “**asking your neighbors for advice**”.

Suppose you move to a new city and want to know if a restaurant is good. You ask your 3 nearest neighbors.

If 2 say “Yes” and 1 says “No,” you decide it's probably good.

KNN uses a similar idea:

- It looks at the **k closest data** points to the new input
- It **classifies or predicts** based on majority vote (for classification) or average (for regression)

Distance Metrics

To find the "nearest neighbors", we calculate distance between data points.

◆ Euclidean Distance (default)

$$d(p, q) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + \dots}$$

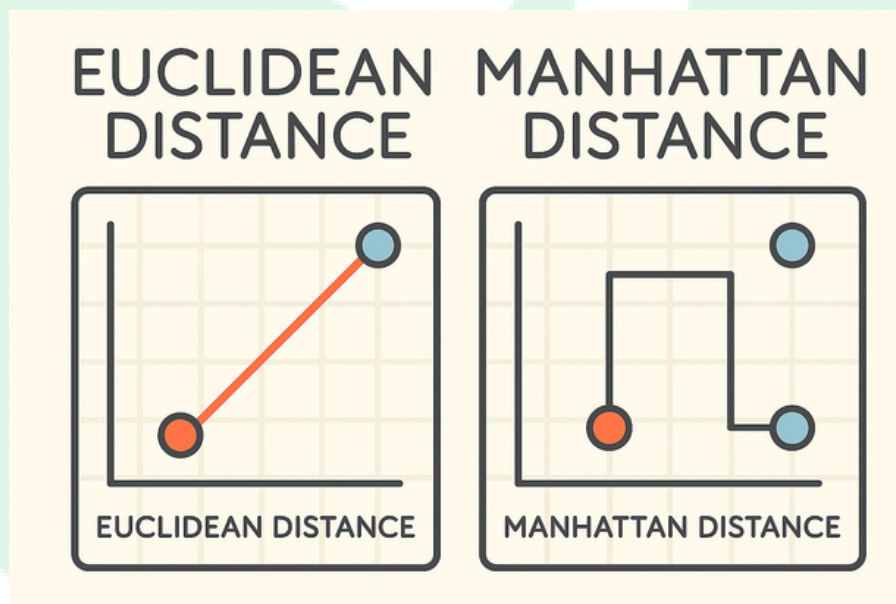
- Think: Straight-line distance (like a ruler)

◆ Manhattan Distance

$$d(p, q) = |x_1 - y_1| + |x_2 - y_2| + \dots$$

- Think: City block distance (like walking around buildings)

You can choose the metric based on your data shape or domain.



Choice of k (Number of Neighbors)

- k = number of neighbors to look at when making a decision.
- Must be a **positive integer**.

Value of k	Result
Too small	Noisy, overfits (high variance)
Too large	Oversmooths, underfits (high bias)
Odd k	Avoids ties in binary classification

- A common practice: Try different values of k and use cross-validation to pick the best.

Lazy Learner Concept

- KNN does not learn a model during training
- Instead, it stores the entire dataset
- At prediction time, it searches through the entire dataset to find neighbors

That's why it's called a lazy learner — it's slow to predict but quick to train.

 Contrast this with Logistic Regression or Decision Trees which build a model during training — those are eager learners.

Recap

- KNN classifies a new data point based on the majority of k nearest neighbors.
- It depends heavily on:
 - The choice of k
 - The scale of the data (so always normalize or standardize!)

code

```

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)

# Train KNN model with k=3
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

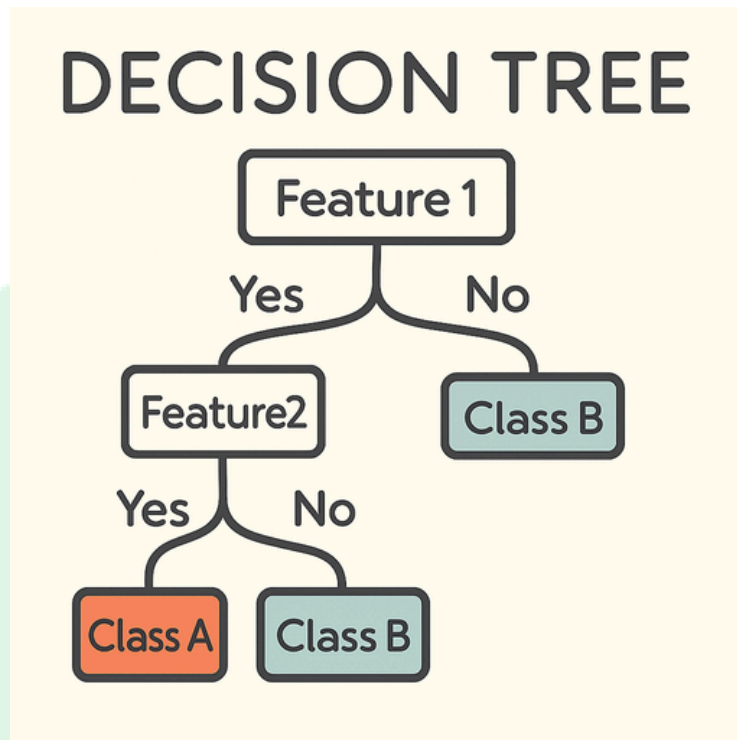
# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))

```

Summary Table

Feature	KNN
Type	Supervised Learning
Model Type	Instance-based / Lazy Learner
Works For	Classification & Regression
Distance Metrics	Euclidean (default), Manhattan, Minkowski
Key Hyperparameter	k – number of neighbors
Pros	Simple, no training time
Cons	Slow at prediction, sensitive to outliers
Use Cases	Handwriting recognition, recommendation systems, medical diagnosis

Decision Tree



A Decision Tree is a supervised learning algorithm used for both classification and regression tasks.

- It splits the dataset into subsets based on feature values.
- Each split is chosen to make the resulting subsets as pure as possible (i.e., containing mostly one class).
- The final decision is made at a leaf node.

🚶 Simple Explanation:

A Decision Tree is like a flowchart that splits your data into smaller and smaller groups by asking yes/no questions, until it makes a decision.

Imagine a game of "20 Questions":

"Is it an animal?" → "Is it a mammal?" → "Does it have fur?" → "Is it a cat?"

Each question narrows down the options – that's how a decision tree works.

Splitting Criteria: Gini Impurity & Entropy

To decide the best split, decision trees use impurity measures.

◆ A. Gini Impurity

Gini measures the probability of a wrong classification if we randomly pick a label from the subset.

$$Gini = 1 - \sum_{i=1}^C p_i^2$$

Where:

- p_i = probability of class i
- C = number of classes

🧠 Interpretation:

- Gini = 0 → pure node (only one class)
- Gini = 0.5 → mixed node (e.g., 50-50 split)

◆ B. Entropy and Information Gain

- Entropy is borrowed from information theory – it measures disorder.

$$Entropy = - \sum_{i=1}^C p_i \cdot \log_2(p_i)$$

- **Information Gain** = How much uncertainty is reduced after a split:

$$IG = Entropy_{parent} - \text{weighted average of } Entropy_{children}$$

🧠 Entropy = 0 → pure

- Entropy = 1 → maximum disorder (e.g., 50-50 split)

✓ Gini is faster to compute and used in CART (Classification and Regression Trees).

✓ Entropy is used in ID3 / C4.5 algorithms.

Pruning Techniques

Trees can easily overfit the training data. To avoid this, we prune the tree to make it smaller and more general.

◆ A. Pre-pruning (Early Stopping)

- Stop tree growth **early**, based on:
 - Max depth
 - Minimum samples required to split
 - Minimum impurity decrease

◆ B. Post-pruning (Reduced Error Pruning)

- Build the full tree first
- Then **cut back** branches if they don't improve performance on **validation data**

 Think of **pre-pruning** as "preventing overgrowth"
 And **post-pruning** as "cutting the dead branches"

code

```

from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt

# Load data
iris = load_iris()
X, y = iris.data, iris.target

# Train decision tree
model = DecisionTreeClassifier(criterion='gini', max_depth=3)
model.fit(X, y)

# Visualize the tree
plt.figure(figsize=(12, 6))
plot_tree(model, feature_names=iris.feature_names, class_names=iris.target_names, filled=True)
plt.show()
  
```


Summary

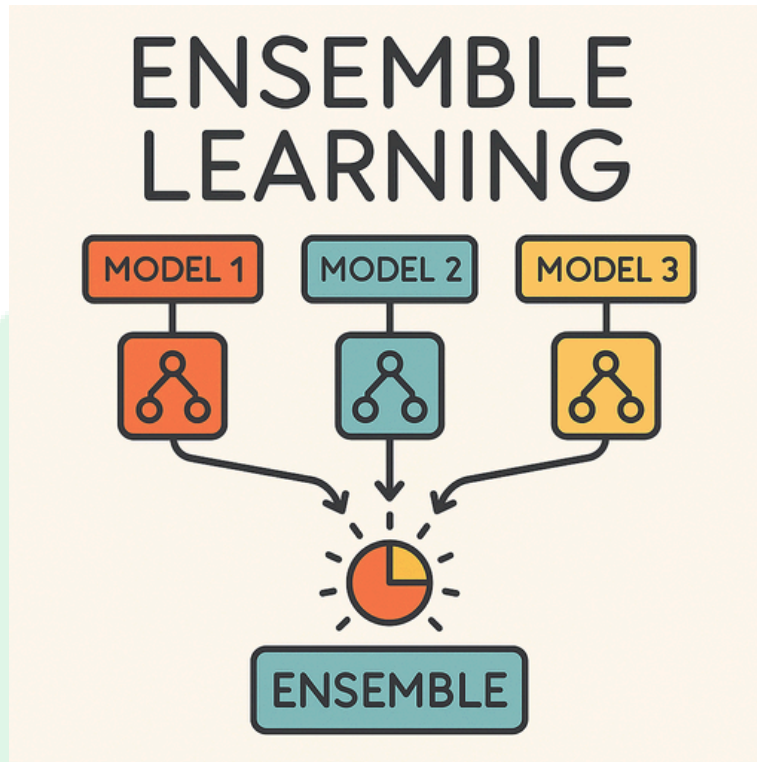
Feature	Explanation
Type	Supervised, Classification & Regression
Structure	Tree (nodes & branches)
Splitting Criteria	Gini, Entropy
Grows by	Finding best feature & value to split data
Pruning	Avoids overfitting by trimming the tree
Pros	Interpretable, handles mixed types
Cons	Overfits easily without pruning



Quick Recap

- Decision Trees **split** data into smaller groups using yes/no questions.
- Splits are based on **impurity** (Gini or Entropy).
- Trees can be **overly complex** unless you prune them.
- You can **visualize** decision trees, making them one of the most **interpretable** ML models.

What is Ensemble Learning?



Ensemble learning is a machine learning paradigm where multiple base models (often weak learners) are trained and combined to solve the same problem and improve performance.

Common strategies:

- Bagging (Bootstrap Aggregating)
- Boosting
- Stacking (advanced ensemble)

✓ Simple Explanation:

“Don’t rely on just one opinion — ask a group!”

Ensemble learning is a technique where we **combine multiple models** to produce a **stronger, more accurate model**.

Just like asking multiple doctors for a diagnosis, ensembles **reduce errors** and **increase reliability**.

BAGGING (Bootstrap Aggregating)

- We create multiple subsets of the original dataset using bootstrapping (random sampling with replacement).
- Each model (usually a decision tree) is trained on a different subset.
- For classification: Take majority vote.
- For regression: Take average prediction.

Goal:

- Reduce variance
- Handle overfitting

Simple Explanation:

Imagine asking 10 different people to solve the same problem independently and then averaging their answers.

Each person sees a slightly different set of data (due to random sampling), so they might make different mistakes — but together, they cancel out errors.

Real Example: Random Forest

- A Random Forest is an ensemble of Decision Trees built using Bagging.
- Each tree sees a random subset of:
 - Data samples
 - Features

BOOSTING

- Models are trained one after another.
- Each new model tries to fix the errors of the last model.
- Final prediction is a weighted combination of all models.

Goal:

- Reduce variance
- Handle overfitting

Simple Explanation:

Think of Boosting like a relay race — each model learns from the mistakes of the previous one.

We train models sequentially, and each new model focuses on the errors made by the previous one. They “boost” the overall performance together.

Real Example: Random Forest

Algorithm	Description
AdaBoost	Adjusts weights of data points after each model
Gradient Boosting	Trains new models on the gradient of the loss
XGBoost / LightGBM / CatBoost	Fast, powerful boosting libraries

Bagging vs Boosting

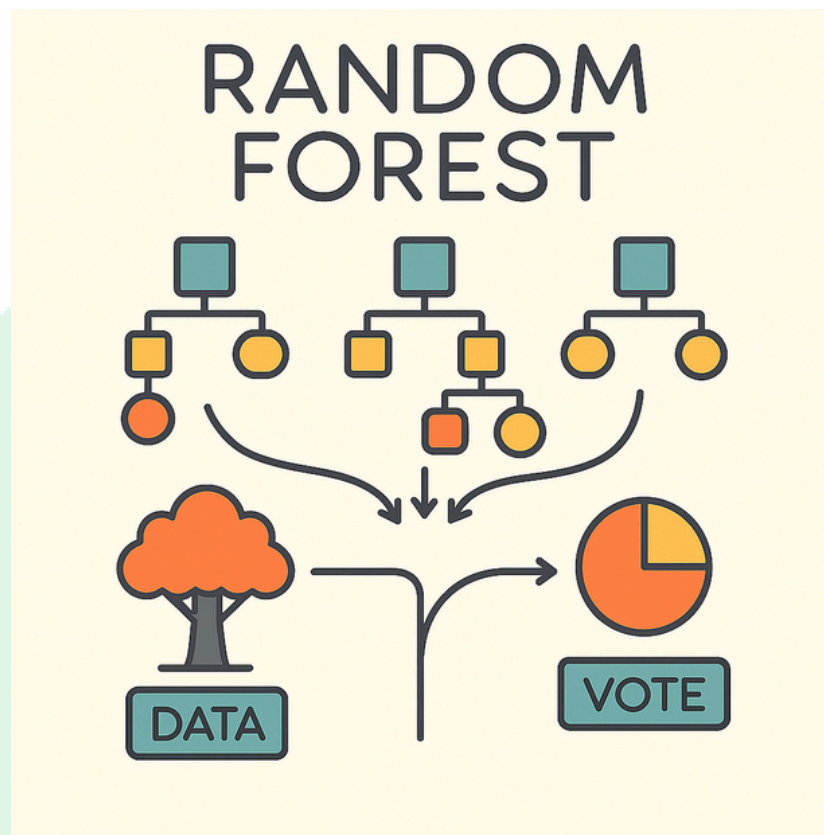
Feature	Bagging	Boosting
Model Training	Parallel	Sequential
Focus	Reduce variance	Reduce bias
Data Sampling	Random subsets (with replacement)	Full data, but reweighted
Example Models	Random Forest	AdaBoost, Gradient Boosting
Overfitting	Less prone	More prone (but tunable)
Speed	Faster (parallelizable)	Slower (sequential)



Recap

- Ensemble Learning combines multiple models for better performance.
- Bagging: Train models independently, combine results (like a team vote).
- Boosting: Train models sequentially, improve by fixing previous errors.
- Both reduce different types of error:
- Bagging → Variance
- Boosting → Bias

Random Forest



- Random Forest is an ensemble algorithm that uses Bagging (Bootstrap Aggregating) and Decision Trees as base learners.
- It builds many decision trees on random subsets of data and features.
- For classification: uses majority voting.
- For regression: uses average prediction.

✓ Simple Explanation:

A Random Forest is like a crowd of decision trees where each tree gives a vote, and the forest decides the majority answer.

Just like asking multiple doctors for a diagnosis — each sees different symptoms (features) and patients (data samples), and their collective opinion is usually better than one alone.

Built-in Bagging Technique:

Random Forest = Bagging + Random Feature Selection

Steps:

1. Bootstrap Sampling: Randomly sample data with replacement to train each tree.
2. Random Feature Selection: Each tree only sees a subset of features at each split.
3. Build multiple decision trees.
4. Aggregate predictions:
 - Classification → Majority vote
 - Regression → Mean of outputs


 This reduces variance and improves generalization.



Feature Importance

Random Forest can automatically tell you which features are most useful for prediction!

How?

Each tree uses some features to split data. Features that:

- Reduce impurity the most across all trees
 - Appear more often near the top of the tree
- ... are considered more important.

Why is it useful?

- Helps in feature selection
- Improves model explainability

Code

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_iris

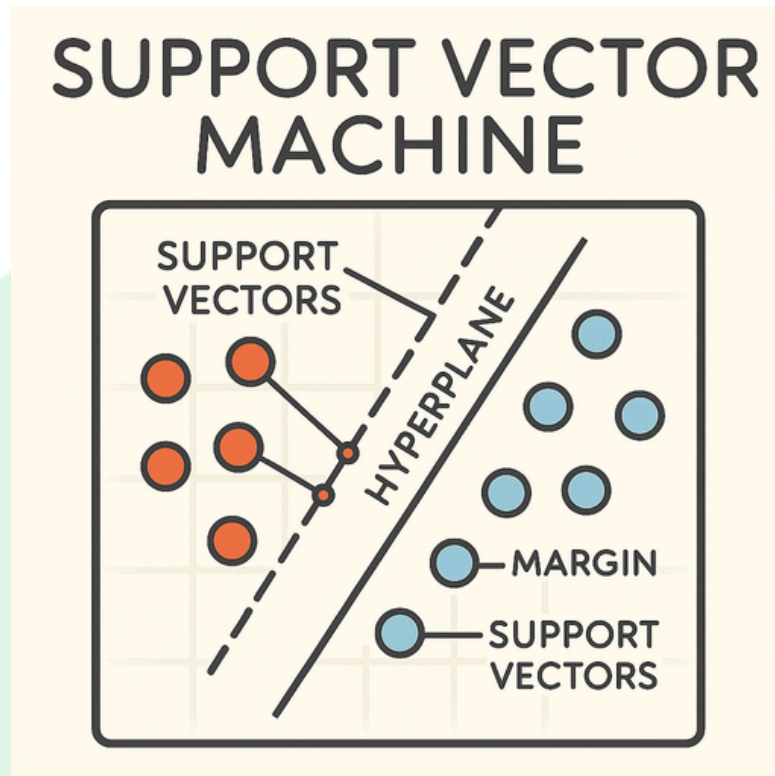
# Load dataset
X, y = load_iris(return_X_y=True)
feature_names = load_iris().feature_names

# Train model
model = RandomForestClassifier()
model.fit(X, y)

# Feature importances
importances = model.feature_importances_
forest_importances = pd.Series(importances, index=feature_names)

# Plot
forest_importances.sort_values().plot(kind='barh', title='Feature Importance')
plt.xlabel('Importance')
plt.show()
```


Support Vector Machines (SVM)



SVM is a supervised learning algorithm that finds the optimal hyperplane that maximizes the margin between two classes.

- The support vectors are the data points closest to the hyperplane.
- The margin is the distance between the hyperplane and the nearest support vectors from both classes.
- SVM tries to maximize this margin to improve generalization.

✓ Simple Explanation

Think of SVM like drawing a line (or hyperplane) that best separates two categories, with maximum space (margin) between them.

If you're drawing a line to separate cats 🐱 and dogs 🐶, SVM finds the one that gives most space from both sides — so it makes confident decisions.

Kernels: Making SVM Work for Non-Linear Data

Kernels let SVM transform data into higher dimensions, so it can find a separating hyperplane even when the data is not linearly separable.

Common Kernels:

Kernel	Use Case
Linear	When data is linearly separable
Polynomial	Adds interaction terms, handles curved boundaries
RBF (Radial Basis Function)	Handles highly complex boundaries

Code:

```

from sklearn.svm import SVC
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

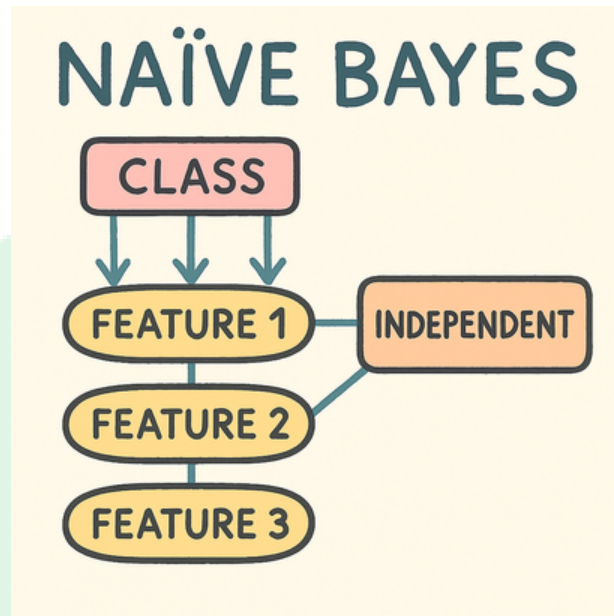
# Create data
X, y = make_classification(n_samples=200, n_features=2, n_classes=2, random_state=42)

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y)

# Train SVM
model = SVC(kernel='rbf', C=1.0)
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
  
```

Naive Bayes



Naive Bayes is a probabilistic classifier based on Bayes' Theorem:

$$P(Y | X) = P \frac{P(X|Y) \cdot P(Y)}{P(X)}$$

Where:

- Y = class (e.g., spam or not)
- X = input features

“Naive” Assumption:

It assumes features are independent given the class label.
This makes computation very simple and fast.

✓ Simple Explanation

Imagine you're guessing if a person has a cold based on:

- Sneezing?
- Fever?
- Headache?

Naive Bayes asks: “What’s the probability of Cold given these symptoms?” using Bayes’ Theorem and assumes symptoms are independent.

Types of Naive Bayes Classifiers

Type	When to Use
Gaussian NB	Features are continuous and follow a normal distribution
Multinomial NB	Features are counts (e.g., word counts in text)
Bernoulli NB	Features are binary (0 or 1 – like presence/absence of words)

Summary – Naive Bayes

Concept	Summary
Foundation	Bayes' Theorem
Assumption	Features are independent
Speed	Very fast and simple
Common Use Cases	Spam filtering, text classification
Pros	Works well with high-dimensional data
Cons	Independence assumption often violated in real data

Code

```

from sklearn.naive_bayes import GaussianNB
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load data
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y)

# Train Naive Bayes
model = GaussianNB()
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))

```



Recap

Feature	Support Vector Machine (SVM)	Naive Bayes
Type	Margin-based classifier	Probabilistic classifier
Assumptions	No specific feature assumptions	Features are independent
Works well when	Clear boundary exists	High-dimensional, sparse data (e.g. text)
Key Concepts	Margin, Kernels, Soft Margin, Support Vectors	Bayes' Theorem, Prior & Likelihood
Speed	Medium (can be slow on large datasets)	Very fast
Interpretability	Medium	High (shows probabilities)

XGBoost (Extreme Gradient Boosting)

✓ Why it's special:

- Regularized Gradient Boosting (prevents overfitting)
- Parallelization + Tree pruning
- Handles missing data well

🔍 Highlights:

- Supports L1 and L2 regularization
- Uses second-order derivatives (Hessian) to improve optimization
- Fast and scalable for large datasets

Code

```
from xgboost import XGBClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load data
X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y)

# Train model
model = XGBClassifier(use_label_encoder=False, eval_metric='logloss')
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

LightGBM (Light Gradient Boosting Machine)

✓ Why it's special:

- Faster than XGBoost on large datasets
- Uses Histogram-based splitting (binned values)
- Grows tree leaf-wise instead of level-wise

🔍 Highlights:

- Very memory efficient
- Can handle categorical features directly
- Better for high-dimensional, sparse datasets

Code

```

import lightgbm as lgb
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y)

model = lgb.LGBMClassifier()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
  
```


CatBoost (Categorical Boosting)

Why it's special:

- Handles categorical variables natively (no encoding needed)
- Less parameter tuning required
- Performs well with default settings

Highlights:

- Uses ordered boosting to avoid overfitting
- Works great with imbalanced and categorical data

Code

```

from catboost import CatBoostClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y)

model = CatBoostClassifier(verbose=0) # suppress training output
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
  
```


Final Comparison

Feature	XGBoost	LightGBM	CatBoost
Speed	Fast	Very fast	Medium
Categorical	Manual encoding needed	Manual or auto encoding	Native support 
Performance	High	High	High
Strength	Regularization + Optimization	Memory & Speed	Works great out-of-the-box
Handles Missing	Yes	Yes	Yes



When to Use What?

Scenario	Use
Large dataset, numeric features	LightGBM
Mixed features, categorical + small data	CatBoost
Tunable + high control	XGBoost

Evaluation Metrics for Classification Models

MODULE

5

Confusion Matrix

✓ Simple Explanation:

A confusion matrix is a table that shows how many correct and incorrect predictions your model made.

It helps you answer:

- Did I classify positives as positives?
- Did I wrongly classify negatives as positives?

Structure:

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)



Example:

Let's say we're detecting if an email is spam:

	Predicted: Spam	Predicted: Not Spam
Actual: Spam	80 (TP)	20 (FN)
Actual: Not Spam	10 (FP)	90 (TN)

Accuracy, Precision, Recall, F1-Score

Metric	Formula	Meaning
Accuracy	$\frac{TP + TN}{TP + TN + FP + FN}$	Overall correctness
Precision	$\frac{TP}{TP + FP}$	How many predicted positives were right
Recall	$\frac{TP}{TP + FN}$	How many actual positives were found
F1-Score	$2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$	Balance between Precision & Recall



Analogy:

Imagine a spam filter:

- Precision: Of all emails marked as spam, how many were actually spam?
- Recall: Of all real spam emails, how many did you catch?

ROC Curve and AUC

- ROC (Receiver Operating Characteristic) curve shows how well your model distinguishes between classes.
- It plots:
 - True Positive Rate (Recall) vs. False Positive Rate

Terms:

Term	Formula
True Positive Rate	$\frac{TP}{TP + FN} = Recall$
False Positive Rate	$\frac{FP}{FP + TN}$

AUC – Area Under the Curve:

- AUC ranges from 0 to 1.
- Higher is better:
 - 1.0 = perfect classifier
 - 0.5 = random guessing
 - < 0.5 = worse than random

Precision-Recall Curve

✓ Simple Explanation:

- Used especially when classes are imbalanced (e.g., 1% fraud, 99% normal).
- Shows trade-off between:
 - Precision (how many predicted positives are correct)
 - Recall (how many actual positives are captured)

💡 When to use:

- ROC is good for balanced datasets
- Precision-Recall Curve is better for imbalanced datasets

Code:

```

from sklearn.metrics import precision_recall_curve
import matplotlib.pyplot as plt

precision, recall, _ = precision_recall_curve(y_test, y_proba)

plt.plot(recall, precision)
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.title("Precision-Recall Curve")
plt.grid()
plt.show()
  
```



Summary Table

Metric	Best Use Case	Notes
Accuracy	Balanced datasets	Misleading on imbalanced data
Precision	Avoid false positives (e.g. email spam)	How precise your positive predictions are
Recall	Avoid false negatives (e.g. cancer detection)	How well you're catching real positives
F1-Score	Balance between Precision & Recall	Useful when both matters
ROC Curve	Binary classification	Shows ability to rank positives vs negatives
AUC	Summary of ROC	Higher AUC = better performance
PR Curve	Imbalanced data	Focuses only on positive class performance

Data Preparation for Supervised Learning

MODULE

6

1. Data Preprocessing

1.1 Handling Missing Values

✓ Simple Explanation:

Sometimes data is incomplete. We must either fill (impute) or remove the missing values.

Method	Use When...
Drop rows	Few missing values, safe to ignore
Mean/Median	Numerical columns, no extreme outliers
Mode	For categorical values
Forward/Backward Fill	For time-series or ordered data

```

import pandas as pd
df = pd.read_csv("data.csv")

# Drop rows with any missing values
df.dropna(inplace=True)

# Fill numerical column with mean
df['age'].fillna(df['age'].mean(), inplace=True)

# Fill categorical with mode
df['gender'].fillna(df['gender'].mode()[0], inplace=True)
  
```

1.2 Encoding Categorical Variables

✓ Simple Explanation:

Machine learning models only work with numbers, so we need to convert text data to numeric form.

Method	Use When...
Label Encoding	Ordinal categories (Low < Medium < High)
OneHot Encoding	Nominal categories (Color: Red, Blue)
Target Encoding	When category affects the target value

```
from sklearn.preprocessing import LabelEncoder, OneHotEncoder

# Label encoding
le = LabelEncoder()
df['size'] = le.fit_transform(df['size']) # Small=2, Medium=1, Large=0

# OneHot encoding
df = pd.get_dummies(df, columns=['color'], drop_first=True)

# Target Encoding (custom)
mean_target = df.groupby('city')['price'].mean()
df['city_encoded'] = df['city'].map(mean_target)
```

1.3 Feature Scaling

✓ Simple Explanation:

Some models (like logistic regression or KNN) are sensitive to large numbers. Scaling makes sure all features are on the same scale.

Scaler	Scales To	Use Case
MinMax	[0, 1]	Normalizes data
Standard	mean=0, std=1	Keeps Gaussian shape, useful in LR
Robust	Uses median/IQR	Good for outliers

```

from sklearn.preprocessing import MinMaxScaler, StandardScaler, RobustScaler

scaler = StandardScaler()
df[['age', 'salary']] = scaler.fit_transform(df[['age', 'salary']])
  
```

1.4 Train-Test Split and Stratification

✓ Simple Explanation:

We divide the data into:

- Training set (to learn)
- Testing set (to evaluate)

Stratification ensures that the class distribution is preserved (especially useful for classification).

Scaler	Scales To	Use Case
MinMax	[0, 1]	Normalizes data
Standard	mean=0, std=1	Keeps Gaussian shape, useful in LR
Robust	Uses median/IQR	Good for outliers

```

from sklearn.model_selection import train_test_split

X = df.drop('target', axis=1)
y = df['target']

# Stratified split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)
  
```

2. Feature Engineering

2.1 Feature Selection

✓ Simple Explanation:

Choosing the most useful features and removing the rest improves model performance and interpretability.

Method	Use When...
Drop rows	Few missing values, safe to ignore
Mean/Median	Numerical columns, no extreme outliers
Mode	For categorical values
Forward/Backward Fill	For time-series or ordered data

```

import pandas as pd
df = pd.read_csv("data.csv")

# Drop rows with any missing values
df.dropna(inplace=True)

# Fill numerical column with mean
df['age'].fillna(df['age'].mean(), inplace=True)

# Fill categorical with mode
df['gender'].fillna(df['gender'].mode()[0], inplace=True)
  
```

2.2 Feature Transformation

✓ Simple Explanation:

Modifying the values of features to improve relationships with the target.

Transformation	Use When...
Log Transform	Skewed data (e.g. salary, income)
Binning	Grouping continuous values into categories
Interaction	Creating new features using 2+ existing ones

```

import numpy as np

# Log transformation
df['log_income'] = np.log1p(df['income'])

# Binning
df['age_bin'] = pd.cut(df['age'], bins=[0, 18, 35, 60, 100], labels=['Teen', 'Young Adult', 'Adult', 'Senior'])

# Interaction feature
df['income_per_age'] = df['income'] / df['age']
  
```

Wrapper Methods

- Uses a model to test feature combinations
- Recursive Feature Elimination (RFE) is common

```

from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()
rfe = RFE(model, n_features_to_select=5)
X_rfe = rfe.fit_transform(X, y)
  
```

Embedded Methods

- Feature selection is built into the model
- Lasso (L1 regularization) shrinks some coefficients to zero

```

from sklearn.linear_model import Lasso
model = Lasso(alpha=0.01)
model.fit(X, y)

# Selected features (non-zero coef)
selected = X.columns[model.coef_ != 0]
  
```

 Summary Table

Task	Tool/Technique
Handle missing data	dropna(), fillna(), mean/mode
Encode categories	LabelEncoder, OneHot, Target map
Feature scaling	StandardScaler, MinMax, Robust
Feature selection	RFE, correlation, Lasso
Feature transformation	log(), cut(), interaction terms
Split dataset	train_test_split() with stratify

Model Training and Tuning

MODULE

7

1. Training Workflow

Fit vs Predict vs Score

✓ Simple Explanation:

Step	What It Does
fit()	Train the model on your training data
predict()	Use the model to make predictions on new data
score()	Evaluate model performance (default = accuracy for classification)

```

from sklearn.linear_model import LogisticRegression
model = LogisticRegression()

model.fit(X_train, y_train)           # Training
y_pred = model.predict(X_test)        # Predicting
accuracy = model.score(X_test, y_test) # Scoring
print("Accuracy:", accuracy)
  
```

Cross-Validation Techniques

Cross-validation helps you check model performance more reliably by splitting the training data multiple times.

K-Fold Cross-Validation

✓ Simple Explanation:

- Split the data into k equal parts (folds)
- Train the model on k-1 folds and test on the remaining fold
- Do this k times, changing the test fold each time
- Average the results

```
from sklearn.model_selection import cross_val_score
from sklearn.tree import DecisionTreeClassifier

scores = cross_val_score(DecisionTreeClassifier(), X, y, cv=5)
print("Average accuracy:", scores.mean())
```

Stratified K-Fold

✓ Simple Explanation:

- Like K-Fold but ensures class ratios are preserved in each fold (important for imbalanced datasets).

```
from sklearn.model_selection import StratifiedKFold
skf = StratifiedKFold(n_splits=5)
```

Leave-One-Out (LOO or LOOCV)

Simple Explanation:

- Train on all data except one sample, test on that one
- Repeat this for each sample

Use When:

- Very small datasets
- But very slow for large data

```

from sklearn.model_selection import LeaveOneOut
loo = LeaveOneOut()
scores = cross_val_score(LogisticRegression(), X, y, cv=loo)
print("LOO Accuracy:", scores.mean())
  
```

Avoiding Data Leakage

What is it?

Data leakage happens when your model sees future or test data during training.

Common Examples:

- Applying scaling before train-test split ❌
- Using test set information during feature engineering ❌

Solution:

- Always do scaling/imputation/encoding inside a pipeline
- Split first, then transform only training, then apply same transform to test

2. Hyperparameter Tuning

Hyperparameters are the settings you choose, not learned by the model (e.g., max_depth in Decision Tree, k in KNN).

Why Tune Them?

Because they hugely impact model performance!

Grid Search

✓ Simple Explanation:

- Tries all combinations of hyperparameters in a grid
- Very exhaustive but slow

```

from sklearn.model_selection import GridSearchCV

param_grid = {'max_depth': [3, 5, 10], 'min_samples_split': [2, 4]}
grid = GridSearchCV(DecisionTreeClassifier(), param_grid, cv=5)
grid.fit(X, y)
print("Best Parameters:", grid.best_params_)
  
```

Randomized Search

✓ Simple Explanation:

- Tries random combinations from parameter ranges
- Faster, good when you have many parameters

```

from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint

param_dist = {'max_depth': randint(1, 10)}
random_search = RandomizedSearchCV(DecisionTreeClassifier(), param_dist, n_iter=10, cv=5)
random_search.fit(X, y)
print("Best Parameters:", random_search.best_params_)
  
```

Early Stopping (esp. in Boosting)

Simple Explanation:

- Stop training if performance doesn't improve after few rounds
- Helps avoid overfitting

```

from xgboost import XGBClassifier

model = XGBClassifier(n_estimators=1000)
model.fit(X_train, y_train, early_stopping_rounds=10, eval_set=[(X_test, y_test)], verbose=True)
  
```

Summary Table

Concept	Tool/Technique
Fit/Predict/Score	.fit(), .predict(), .score()
Cross-validation	KFold, StratifiedKFold, LeaveOneOut
Avoid leakage	Split first, then transform
Grid Search	Exhaustive hyperparameter search
Randomized Search	Random hyperparameter samples
Bayesian Optimization	Optuna / Hyperopt for smart tuning
Early Stopping	XGBoost/LightGBM for stopping early

Bonus Concepts for Depth

MODULE

8

1. Learning Curves (Bias-Variance Visualization)

Fit vs Predict vs Score

✓ Simple Explanation:

Learning curves plot training and validation error as the amount of training data increases. They help us understand:

- If the model is underfitting (high bias) or
- Overfitting (high variance)

Scenario	Training Error	Validation Error	Insight
Underfitting	High	High	Too simple model
Overfitting	Low	High	Too complex model
Good Fit	Low	Low	Balanced model

```

from sklearn.model_selection import learning_curve
from sklearn.ensemble import RandomForestClassifier
import matplotlib.pyplot as plt

train_sizes, train_scores, test_scores = learning_curve(RandomForestClassifier(), X, y, cv=5, train_sizes=np.linspace(0.1, 1.0, 5))

plt.plot(train_sizes, train_scores.mean(axis=1), label='Train')
plt.plot(train_sizes, test_scores.mean(axis=1), label='Validation')
plt.xlabel("Training Size")
plt.ylabel("Score")
plt.legend()
plt.title("Learning Curve")
plt.show()
  
```


2. Curse of Dimensionality

✓ Simple Explanation:

As the number of features (dimensions) increases, the data becomes sparse, making it:

- Harder for models to generalize
- More prone to overfitting
- Slower to train

Why It Matters:

- Distance metrics (e.g., in KNN) become meaningless in high dimensions.
- Model complexity increases.

Solution:

- Feature selection (e.g., Lasso)
- Dimensionality reduction (e.g., PCA)

3. Class Imbalance Techniques

What is Class Imbalance?

✓ Simple Explanation:

In a classification problem, if one class has many more samples than the other(s), we say the dataset is imbalanced.

Example:

Suppose you're building a model to detect fraudulent transactions:

Class	Meaning	Count
0	Not Fraud	98,000
1	Fraud	2,000

Here, the "Not Fraud" class dominates. If a model predicts everything as 0, it gets 98% accuracy — but it's useless because it misses most frauds.

Why It's a Problem?

1. High Accuracy is Misleading – A model might get 98% accuracy just by guessing the majority class.
2. Poor Recall for Minority Class – Fails to detect rare but important outcomes (like fraud or disease).
3. Skewed Learning – Models don't learn minority patterns well.

Techniques to Handle Class Imbalance

1. Resampling Techniques

◆ A. Undersampling (Downsampling)

- Remove samples from the majority class.
- Pros: Faster, simple.
- Cons: May lose important data.

```
from imblearn.under_sampling import RandomUnderSampler

rus = RandomUnderSampler()
X_under, y_under = rus.fit_resample(X, y)
```

◆ B. Oversampling (Upsampling)

- Duplicate samples from the minority class.
- Pros: Doesn't discard data.
- Cons: May overfit due to duplication.

```

from imblearn.over_sampling import RandomOverSampler

ros = RandomOverSampler()
X_over, y_over = ros.fit_resample(X, y)
  
```

◆ C. SMOTE (Synthetic Minority Over-sampling Technique)

- Creates new synthetic examples (not just duplicates) for the minority class.
- Works by interpolating between existing minority class points.

📌 Tip: Use SMOTE when you want to preserve majority data and still balance classes smartly.

```

from imblearn.over_sampling import SMOTE

smote = SMOTE()
X_smote, y_smote = smote.fit_resample(X, y)
  
```

◆ D. SMOTE + Tomek Links / Edited Nearest Neighbors

- Combine oversampling with cleaning the boundary using Tomek links.
- This helps reduce noise and overlap between classes.

```
from imblearn.combine import SMOTETomek

smt = SMOTETomek()
X_resampled, y_resampled = smt.fit_resample(X, y)
```

2. Class Weighting

Instead of changing the data, tell the model to give more importance to the minority class.

📌 **Works with: Logistic Regression, SVM, Random Forest, etc.**

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(class_weight='balanced')
model.fit(X_train, y_train)
```

You can also set custom weights manually:

```
weights = {0: 1, 1: 5}
model = RandomForestClassifier(class_weight=weights)
```



3. Anomaly Detection Approach

If the minority class is very rare, treat it as an anomaly detection problem using:

- Isolation Forest
- One-Class SVM



4. Evaluation Metrics for Imbalanced Data

Use metrics that focus on minority class performance:

Metric	When to Use
Precision	When false positives are costly
Recall	When false negatives are costly
F1-Score	Balance between Precision and Recall
AUC-ROC	Overall separability between classes
PR-AUC	Better for highly imbalanced problems



5. Ensemble with Balanced Sampling

Use models that do built-in resampling, e.g.:

- `BalancedRandomForestClassifier`
- `EasyEnsembleClassifier`

```

from imblearn.ensemble import BalancedRandomForestClassifier

model = BalancedRandomForestClassifier()
model.fit(X_train, y_train)
  
```

Summary Table

Method	Type	When to Use
Undersampling	Data-level	Small data, fast training
Oversampling	Data-level	Risk of overfitting
SMOTE	Data-level	Need synthetic diversity
Class Weights	Model-level	When using built-in classifier
Anomaly Detection	Alternative	Extreme imbalance (e.g., 0.1%)
Evaluation Metrics	Metrics	When accuracy is misleading